Aror University of Art, Architecture, Design & Heritage Sukkur.

Department of Artificial Intelligence, Multimedia Gaming and Cyber Security

Object Oriented Programming (Fall-2025)

LAB No. 6

Objective of Lab No. 6:

After performing lab, students will be able to:

- Implement classes in Java and Create Objects
- Use Default constructor and new operator
- Use Dot Operator
- Modify default and create Parameterized constructor
- Implement void and return type methods

Recap of Last Lab -5:

1. **Create a class circle with attributes radius and color, Now:**
    a. create two objects namely red_circle and green_circle in the main method using default constructor.
    b. Add a method named calculateArea() which calculates the area of calling object and prints it on the console screen, note the method does not return any value and accepts no parameter.
    c. Use dot operator to initialize the instance variables of red_circle and green_circle (Use any radius value)
    d. Now print the radius of both circles (Use Dot Operator with instance variables)

Solution:

```
class Circle {

   double radius;
   String color;
```

```
    Circle() {
    }

    void calculateArea() {
        double area = Math.PI * radius * radius;
        System.out.println("Area of " + color + " circle: " + area);
    }
}

public class Main {

    public static void main(String[] args) {

        Circle red_circle = new Circle();
        Circle green_circle = new Circle();

        red_circle.radius = 5;
        red_circle.color = "Red";

        green_circle.radius = 7;
        green_circle.color = "Green";
        System.out.println("Radius of Red Circle: " + red_circle.radius);
        System.out.println("Radius of Green Circle: " + green_circle.radius);

        red_circle.calculateArea();
        green_circle.calculateArea();
    }
}
```

2. **Now Modify the Task 1 in following way:**
    a. Initialize the instance variables using parameterized constructor (Demonstrate bypassing the local variable hiding)
    b. calculateArea() method returns the area of calling object, choose return type wisely.
    c. Print the radius of both circles with the help of getter methods for color and radius.

Solution:

```
class Circle {
    private double radius;
```

```
    private String color;

    Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    double calculateArea() {
        return Math.PI * radius * radius;
    }

    double getRadius() {
        return radius;
    }

    String getColor() {
        return color;
    }
}

public class Main {

    public static void main(String[] args) {

        Circle red_circle = new Circle(5, "Red");
        Circle green_circle = new Circle(7, "Green");

       System.out.println("Radius of " + red_circle.getColor() +
                " circle: " + red_circle.getRadius());

        System.out.println("Radius of " + green_circle.getColor() +
                " circle: " + green_circle.getRadius());

        System.out.println("Area of " + red_circle.getColor() +
                " circle: " + red_circle.calculateArea());

        System.out.println("Area of " + green_circle.getColor() +
                " circle: " + green_circle.calculateArea());
    }
}
```

3. **Create a class called Bank Account with attributes balance and AccountNumber, Now:**
    a. Create two Objects namely acc1, acc2 with parameterized constructor, by bypassing local variable hiding (Initial balance of account 1 is 1000 and account 2 is 500).
    b. Implement methods to deposit, withdraw and check balance of an account.
    c. Deposit 500 in account 1 and 1500 in account 2 with the help of method.
    d. Check balance of both accounts now.
    e. Withdraw 500 from account 1 (Make sure you have that much balance…)
    f. Check balance of account 1.
    g. Withdraw 3000 from account 2.

Solution:

```
class BankAccount {
      private double balance;
   private String accountNumber;

   BankAccount(String accountNumber, double balance) {
      this.accountNumber = accountNumber;
      this.balance = balance;
   }
   void deposit(double amount) {
      if (amount > 0) {
         balance += amount;
         System.out.println(amount + " deposited in Account " + accountNumber);
      } else {
         System.out.println("Invalid deposit amount.");
      }
   }
   void withdraw(double amount) {
      if (amount <= balance) {
         balance -= amount;
         System.out.println(amount + " withdrawn from Account " + accountNumber);
      } else {
         System.out.println("Insufficient balance in Account " + accountNumber);
      }
   }
```

```
    void checkBalance() {
        System.out.println("Balance of Account " + accountNumber + ": " + balance);
    }
}
public class Main {

    public static void main(String[] args) {


        BankAccount acc1 = new BankAccount("ACC101", 1000);
        BankAccount acc2 = new BankAccount("ACC102", 500);

        acc1.deposit(500);
        acc2.deposit(1500);
        acc1.checkBalance();
        acc2.checkBalance();
        acc1.withdraw(500);
        acc1.checkBalance();
        acc2.withdraw(3000);
        acc2.checkBalance();
    }
}
```

4. Write a program that would print the information (name, year of joining, salary, address) of three employees by creating a class named 'Employee'. The output should be as follows:

Name	Year_of_Joining	Address
Robert	1994	WallsStreat
Sam	2000	WallsStreat
John	1999	WallsStreat

Solution:

```
class Employee {

    String name;
    int yearOfJoining;
    double salary;
```

```
    String address;

    Employee(String name, int yearOfJoining, double salary, String address) {
        this.name = name;
        this.yearOfJoining = yearOfJoining;
        this.salary = salary;
        this.address = address;
    }
    void display() {
        System.out.println(name + "\t\t" + yearOfJoining + "\t\t\t" + address);
    }
}

public class Main {

    public static void main(String[] args) {
        Employee emp1 = new Employee("Robert", 1994, 50000, "WallsStreet");
        Employee emp2 = new Employee("Sam", 2000, 60000, "WallsStreet");
        Employee emp3 = new Employee("John", 1999, 55000, "WallsStreet");
        emp1.display();
        emp2.display();
        emp3.display();
    }
}
```

5. **Create a class student having attributes id, name and grades (array of 5 double elements)**
    a. Initialize the above given attributes with the help of parameterized constructor by creating 2 objects.
    b. Add a method display_average_grade() which displays the average grade of calling object.
    c. Add a method calc_percentage() which converts each numeric grade into percentage form and returns the array of percentages, remember maximum marks for each grade are 200.
    d. Add a method called concat_id_name, which concatenates the id and name of student and returns the concatenated result.
    e. Demonstrate the use of all the above methods by calling them over the objects.

Solution:

```
class Student {
```

```
    private int id;
    private String name;
    private double[] grades;   // Array of 5 elements


    Student(int id, String name, double[] grades) {
        this.id = id;
        this.name = name;
        this.grades = grades;
    }

    void display_average_grade() {
        double sum = 0;
        for (int i = 0; i < grades.length; i++) {
            sum += grades[i];
        }
        double average = sum / grades.length;
        System.out.println("Average grade of " + name + " : " + average);
    }

    double[] calc_percentage() {
        double[] percentages = new double[grades.length];
        for (int i = 0; i < grades.length; i++) {
            percentages[i] = (grades[i] / 200) * 100;
        }
        return percentages;
    }


    String concat_id_name() {
        return id + "_" + name;
    }
}

public class Main {

    public static void main(String[] args) {

        double[] grades1 = {180, 150, 170, 160, 190};
        double[] grades2 = {140, 130, 150, 120, 110};
```

```
        Student s1 = new Student(101, "Ali", grades1);
        Student s2 = new Student(102, "Sara", grades2);

        s1.display_average_grade();
        s2.display_average_grade();
        double[] percent1 = s1.calc_percentage();
        double[] percent2 = s2.calc_percentage();

        System.out.println("\nPercentages of Ali:");
        for (double p : percent1) {
            System.out.println(p + "%");
        }
        System.out.println("\nPercentages of Sara:");
        for (double p : percent2) {
            System.out.println(p + "%");
        }

        System.out.println("\nConcatenated ID and Name:");
        System.out.println(s1.concat_id_name());
        System.out.println(s2.concat_id_name());
    }
}
```

6. **Create a class matrix, with attributes number of rows and number of columns, and a matrix of size rows and columns named matx:**
    a. Add a constructor which sets the number of rows and number of columns when an object is created.
    b. Add a get_matrix() method which displays the elements of matt of a matrix object
    c. Add a method called set_element() which accepts row number and column number along with the value, and sets the value at that row number and column number.
    d. Create an object matrix1 having 4 rows and 3 columns.
    e. Create an object matrix2 having 3 rows and 3 columns.
    f. Initialize the matt attribute of both matrices.
    g. Update the element of matt of matrix one at row number 1 and column number 2 with 3.
    h. Call get_matrix() on both objects.

Solution:

```
class Matrix {
```

```
    private int rows;
    private int cols;
    private int[][] matx;


    Matrix(int rows, int cols) {
        this.rows = rows;
        this.cols = cols;
        matx = new int[rows][cols];   // initialize matrix size
    }

    void get_matrix() {
        System.out.println("Matrix (" + rows + "x" + cols + "):");

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(matx[i][j] + " ");
            }
            System.out.println();
        }
        System.out.println();
    }

    void set_element(int row, int col, int value) {
        if (row >= 0 && row < rows && col >= 0 && col < cols) {
            matx[row][col] = value;
        } else {
            System.out.println("Invalid index position!");
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Matrix matrix1 = new Matrix(4, 3);
        Matrix matrix2 = new Matrix(3, 3);
        int value = 1;
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 3; j++) {
                matrix1.set_element(i, j, value++);
            }
        }
```

```
        value = 10;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                matrix2.set_element(i, j, value++);
            }
        }
        matrix1.set_element(1, 2, 3);
        matrix1.get_matrix();
        matrix2.get_matrix();
    }
}
```

Lab Exercises:

1. **Create a class called Circle with a private instance variable radius.**
    a. **Provide public getter and setter methods for radius, but ensure that the setter method validates that the radius is always positive.**
    b. **Create objects of Circle class and show the usage of setter and getter methods.**

2. **Create a class called BankAccount with private instance variables accountNumber and balance.**
    a. Ensure that accountNumber is a read-only variable and balance can be accessed and modified using getter and setter methods.
    b. Create two objects of this class, if the user does not provide any information during creation of the object, then make accountNumber as -1 and let balance remain 0, but if the user provides complete information then set the accountNumber and balance with the information provided by user.

3. **Create a class called Employee with private instance variables id, name, and salary. Make this class as a read-only class**

    a. Include a method called raiseSalary that takes a double amount as a parameter and increases the salary by that amount. Ensure that the raiseSalary method cannot be accessed from outside the class.
    b. Create objects of this class to show it's functionality.

4. **Create a class called StringUtils with overloaded methods: to concatenate two strings, concatenate three strings, and concatenate an array of strings. The methods should return the concatenated string.**

5. **Create a class called DateUtils with overloaded methods to format a date in three different ways (e.g., dd/mm/yyyy, mm/dd/yyyy, yyyy/mm/dd). The methods should return the formatted**

date as a string.

Create a class called MathUtils with overloaded methods to calculate the square root of an integer, double, and float. The methods should